MedInfo Assistant - Project Narrative

Generated 2026-10-15 from narrative.md - information only, not medical advice.

Contents

1. Executive summary
2. Business problem and objectives
3. Target users and use cases
4. Functional scope and non-functional requirements
5. End-to-end user journey (upload -> analyze -> review evidence -> generate report)
6. Frontend and backend responsibilities
7. Agent responsibilities and orchestration
8. RAG, source retrieval and citation verification
9. Data flow and storage decisions
10. LLM provider selection and fallback
11. Security, privacy and medical safety
12. Repository structure and key modules
13. Development journey
14. Local setup and Docker demonstration
15. Testing and evaluation
16. Observability and failure handling
17. AWS deployment roadmap (Planned)
18. Architecture trade-offs and ADR references
19. Known limitations and future enhancements
20. Interview-ready explanation (2 minutes)

> **Status labels used throughout: Implemented** (built, covered by offline tests) · **Verified live** (exercised against real services, transcript recorded on 2026-10-14) · **Planned** (designed or written, not built or not run). Nothing here claims clinical validation, accuracy or regulatory approval. All data is SYNTHETIC.

1. Executive summary

MedInfo Assistant is a safety-first medical **information and research** assistant. A set of narrow agents - triage, scope, retrieval/re-ranking, grounded answering, verification and (for clinicians) a ReAct research agent - runs in a fixed, auditable order. Every answer comes only from trusted, cited sources; urgent symptoms get emergency advice first; it never diagnoses or prescribes; uploads and web pages are treated as untrusted data; personal health information is redacted before anything leaves the server. Requirements: docs/01-PRD.md.

2. Business problem and objectives

Medical questions are high-stakes and time-pressured. General chatbots answer from memory without sources, can be confidently wrong, may diagnose or suggest doses, and rarely escalate emergencies. Research analysis across guidelines, drug labels and papers is slow, and versions conflict. Uploaded case notes contain PHI; documents and web pages can carry hidden instructions.

Objectives: (1) grounded, cited answers or an honest "not covered"; (2) emergency escalation first; (3) no diagnosis or prescribing; (4) privacy by design; (5) measurable safety with release gates; (6) a delivery path that is secure by default.

3. Target users and use cases

User	Use cases
Public user	Understand a condition, a test, what a label says; get emergency direction; keep or delete history
Clinician	Guideline/literature questions, research mode with allow-listed web sources, review sampled answers
Admin	Curate the trusted corpus with provenance, monitor evaluation and review results

4. Functional scope and non-functional requirements

Functional scope (status in the PRD table): triage, scope classes, grounded answers, post-answer checks, trusted corpus + private uploads, OCR, incremental ingestion, hybrid retrieval, research agent, memory, evidence reports, clinician review, evaluation. Non-functional: mandatory safety pipeline, local PHI redaction, JWT/RBAC, strict CSP, fallback + circuit breakers, observability, non-root read-only containers. See docs/01-PRD.md sections 5-6.

5. End-to-end user journey (upload -> analyze -> review evidence -> generate report)

1. **Sign in** (Supabase email/password; role from `app_metadata`).
2. **Upload** (optional): a user's own document is OCR'd if scanned, checked for hidden instructions, PHI-redacted, chunked and embedded into a private collection. Admins ingest trusted, allow-listed sources.
3. **Analyze:** ask a question; the UI streams real pipeline phases (safety -> searching -> writing -> verifying). Emergencies show a pinned banner first.
4. **Review evidence:** citation chips link to source cards (publisher, page/section, version, date, user-provided/web badges); a "Sources and limitations" panel; version conflicts and stale sources noted; a collapsible "How this answer was produced" panel lists each agent step with its outcome and timing (decisions and counts only, no user text). **Implemented** and covered by API and browser tests.
5. **Generate report:** download a Markdown evidence report of the conversation (questions, answers, sources, limitations, disclaimer). **Implemented** and covered by API and browser tests.

6. Frontend and backend responsibilities

- **Frontend** (`frontend/`): plain HTML/CSS/JS, no build step, strict CSP, text-node-only rendering; served by unprivileged nginx which proxies `/api` . docs/05
- **Backend** (`backend/`): FastAPI with thin routes, the safety pipeline in `ChatService` , LangGraph graphs, provider code isolated in `providers.py` . docs/06, API

7. Agent responsibilities and orchestration

Agent	Does	Cannot
Triage	Rules + temperature-0 LLM; emergency/crisis decision	Write or soften the emergency text
Scope	7-class intent; stricter of rules/LLM wins	Relax a rules decision
Query rewriter	Standalone queries for follow-ups	Change the answer
Retriever + re-ranker	Hybrid search, RRF, LLM selection, newest version first	Add content; fails open to fused order
Answer agent	Writes cited sentences from passages only	Run without passages; skip checks
Verifier	Dosing/diagnosis/citation/number/drug checks; faithfulness judge	Pass an answer it cannot verify (fails closed)
Research agent (ReAct)	Chooses corpus/web tools, max 3 calls	Write the answer; use non-allow-listed sites
Guardrails	Deterministic block/redact/flag at input, document, output	Be weakened on safety-enforced rules

Orchestration is a fixed pipeline (not free-form agent negotiation) - a deliberate trade-off for predictability and auditability. Each agent records a trace step (agent, outcome, detail, status, ms) that is returned with the answer and shown as "How this answer was produced" (**Implemented**, tested offline). docs/03, ADR-002

8. RAG, source retrieval and citation verification

Loaders (PDF per page, DOCX, PPTX, XLSX, CSV/TSV, text, images) with RapidOCR for scans; chunking (recursive 15%, semantic, parent-child 10%, table-aware); content-hash incremental upsert; pgvector cosine + Postgres BM25-style keyword search fused by RRF; LLM re-ranker; metadata filters; newest-version preference. Verification: every sentence must cite a provided passage, every number and known drug name must appear in the passages, doses must be verbatim quotes, high-risk topics need the judge. docs/04

9. Data flow and storage decisions

Supabase Postgres holds vectors (two collections per embedding model), the ingest registry, encrypted conversations, review items and evaluation runs; Redis holds only expiring cache, short-term memory and rate-limit counters. Embedding vectors from different models are never mixed. docs/08

10. LLM provider selection and fallback

OpenAI-compatible chain EURI -> Gemini -> Groq with per-model circuit breakers; EURI embeddings (1536); answers at 0.1-0.3, classifiers/judges at 0. Quota -> 429, provider error -> 502, unavailable -> 503; classifiers fail open, the judge fails closed; the emergency notice survives outages. ADR-001, ADR-005

11. Security, privacy and medical safety

JWKS-verified ES256/RS256 tokens, roles from `app_metadata`, owner-scoped data; local PHI redaction before any egress; app-level encryption of stored messages; no PHI in logs, metrics or audit; prompt-injection defences at input, document and web stages; source traceability on every answer; red-flag rules marked draft pending clinician review. docs/09

12. Repository structure and key modules

`backend/` (app, tests, evals, synthetic data, scripts) · `frontend/` (src, tests, Dockerfile, nginx) · `tests/integration/` · `docs/` (PRD...AWS plan, ADRs, diagrams, screenshots, OpenAPI) · `presentation/` · `deploy/k8s/` · `infra/terraform/` · `monitoring/` · `CLAUDE.md` + `.claude/` (rules, skills, agents).

13. Development journey

PRD -> HLD -> architecture -> LLD -> implementation in eight gated stages (safety skeleton; corpus and documents; retrieval quality; guardrails, memory, caching, resilience; evaluation; observability; UI; delivery), then the backend/frontend split, the evidence report and this documentation set. Live testing found and fixed real issues: inline `LIMITATIONS:` parsing, quotes spanning sentences, OCR dropping lines (tuned `det_unclip_ratio`), a flaky re-ingest (DOCX timestamps), stale browser JavaScript, several UI layout bugs found in browser screenshots, and a `.skip` CSS class collision with the skip link that hid skipped rows in the agent-trace panel (status classes now prefixed `st-`, covered by a browser test).

Independent review. A dedicated `safety-reviewer` agent then audited the code and found real gaps - most importantly that the model-written "limitations" line bypassed the post-answer checks, and that an unknown conversation id could return an error before triage ran. Both were reproduced, fixed and pinned with release-blocking tests (see docs/10).

14. Local setup and Docker demonstration

Verified path (no Docker): `scripts/db_init.py`, `scripts/ingest_corpus.py`, then `uvicorn app.main:app --port 8030` from `backend/`; open http://127.0.0.1:8030. **Docker path (written, not yet built here - Docker Desktop did not start):** `docker compose up --build`, open http://127.0.0.1:8080. Demo script: docs/11.

15. Testing and evaluation

- **Offline (Implemented, green):** 419 backend tests including **240** in the release-blocking safety suite (also run against an adversarial fake model that tries to prescribe and diagnose); 10 Playwright browser tests in Microsoft Edge (including 2 for the agent-trace panel); ESLint; ruff. Golden set (14 questions) with verbatim evidence quotes.
- **Verified live (2026-10-14):** EURI chat and embeddings; Supabase schema (pgvector 0.8.2); ingestion of 11 synthetic documents into 25 chunks (2 via OCR), and a re-ingest that skipped all 11; hybrid retrieval + re-ranking answered a definition question citing both guideline versions, led a chest-pain question with the emergency notice and a citation from the OCR'd scanned page, and said "not covered" for an out-of-corpus question; a dosing question returned the strict refusal (safe, but the quote-only variant was not produced in that run); local server readiness with the database "ok".
- **Verified live (2026-10-15):** full evaluation run logged to MLflow. The safety suite passed 100% in every set (emergency, dosing, diagnosis, injection, PHI, out of corpus). Correctness 0.857, citation accuracy 0.929, faithfulness 1.0, judged by Groq gpt-oss-120b. Two dosing/interaction questions were over-refused (safe, but not helpful). A cross-check with the real `ragas` library was stricter on faithfulness (0.85 vs 1.0). Details: test strategy.
- **Not yet run:** live research agent/Tavily, live Gemini/Groq fallback, Docker builds, CI pipeline. Numbers from evaluation runs are always "offline evaluation on a synthetic test set". docs/10

16. Observability and failure handling

Prometheus metrics (HTTP p50/p95/p99 per route template, model and retrieval latency, triage events, refusals by reason, guardrail events, hallucination flags, cache, circuits, rate limits), Grafana dashboard and alert rules (triage spikes, refusal-rate shifts, safety-check failures, open circuits, high latency), one audit line per request and per action without PHI. Dashboards and alerts are **written and tested as files; not yet running**.

17. AWS deployment roadmap (Planned)

Terraform (validated, never applied) for VPC, EKS with KMS-encrypted secrets and NetworkPolicy, ECR, GitHub OIDC and IRSA, in ap-southeast-2 next to the database; Kubernetes manifests with restricted pods, default-deny policies, HPA/PDB; CI with safety gate, smoke test and Trivy; CD via OIDC. docs/12

18. Architecture trade-offs and ADR references

- Fixed pipeline over autonomous multi-agent negotiation: predictability over flexibility.
- Strict grounding over helpfulness: some answers become refusals (seen live for dosing).
- Plain-JS UI over a framework: smaller attack surface, no build step, more manual DOM code.
- Pattern-based PHI redaction: fast and local, but can miss unusual formats (mitigated by minimisation).
- ADRs: 001 · 002 · 003 · 004 · 005 · 006

19. Known limitations and future enhancements

Implemented	Planned / open
Full safety pipeline, guardrails, hybrid RAG, research agent (fakes), memory, reports, review, evaluation tooling, UI, delivery files	Clinician sign-off of red-flag rules
Live: ingestion, retrieval, answers, emergency path, evaluation run (safety suite 100%)	Live research/Tavily; live provider fallback; fixing dosing over-refusal
Compose/K8s/Terraform/CI written and statically validated	Docker builds, CI run, any AWS deployment
Synthetic corpus	Licensed real sources (WHO/NICE/MedlinePlus candidates), DPAs for PHI
Dosing safety (strict refusal)	More reliable quote-only dosing answers from live models
English only	Multilingual support, region-specific emergency/crisis numbers

20. Interview-ready explanation (2 minutes)

"I built a medical information assistant where safety is the architecture, not a feature. A question first goes through local PHI redaction and deterministic plus LLM triage, so emergencies are answered first with text built in code. A scope agent decides whether it's information, research, diagnosis, dosing, unsafe or out of scope - each with a fixed behaviour. Retrieval is hybrid pgvector plus BM25 in Postgres, fused and re-ranked; the answer agent may only use numbered passages and must cite every sentence; a verifier then checks citations, numbers, drug names and verbatim dose quotes, with a temperature-0 judge on high-risk topics that fails closed. Research mode is a ReAct agent that only gathers evidence from the corpus and allow-listed medical sites. Every answer shows how it was produced, agent by agent. The tests run offline with fakes - 419 backend tests, 240 of them a release-blocking safety suite - and I verified the core path live against Supabase and EURI. The trade-off is that it refuses more often than a general chatbot; in this domain that is the right failure mode."